PNML_Frontend Manual

LaTeX Edition

Martin Krause

December 20, 2025

Contents

Chapter 1

Manual: PNML_Frontend

This project is a web-based **Petri net visualizer and editor** that allows users to create, modify, and simulate complex system models. It features a central **Data Store** that synchronizes the state of *Places*, *Transitions*, and *Arcs* across an interactive **SVG canvas**, an automatic **Graph Layout** engine, and a **Simulation** controller. Users can construct nets via drag-and-drop, import **PNML files**, and run token-based simulations either manually or via a backend service.

1.1 System Architecture Overview

The following diagram shows the main components and their relationships:

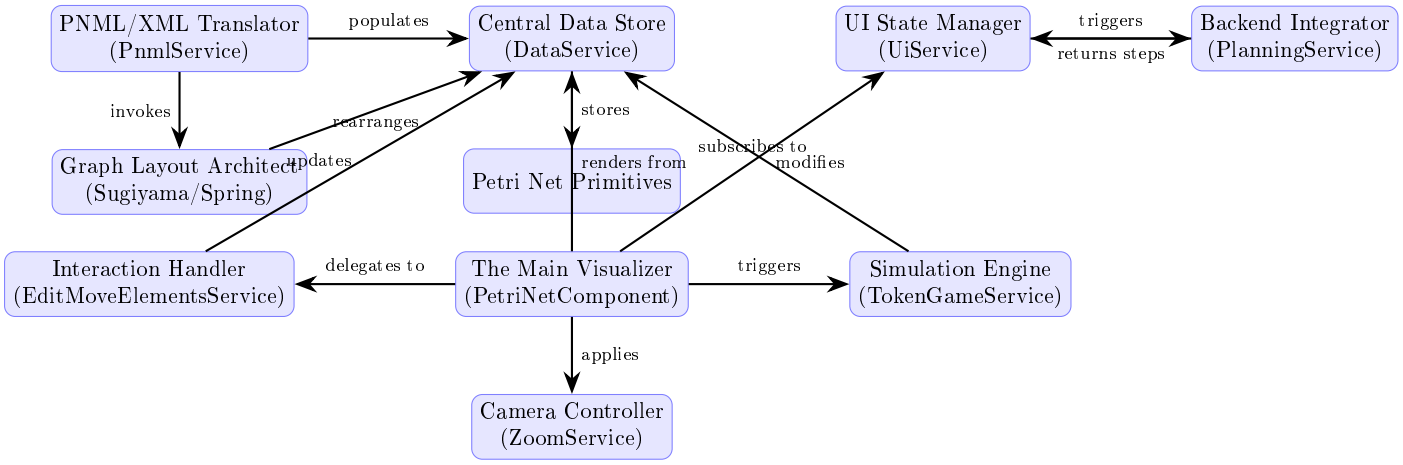

Figure 1.1: PNML_Frontend System Architecture

1.2 Chapters

1. Petri Net Primitives
2. Central Data Store (DataService)
3. The Main Visualizer (PetriNetComponent)
4. PNML/XML Translator (PnmlService)
5. Graph Layout Architect (Sugiyama/Spring Embedder)
6. Interaction Handler (EditMoveElementsService)

7. Camera Controller (ZoomService)
8. UI State Manager (UiService)
9. Simulation Engine (TokenGameService)
10. Backend Integrator (PlanningService)

Chapter 2

Petri Net Primitives

Welcome to the **PNML_Frontend** project! If you are new to Petri nets, don't worry. We are going to start with the absolute basics.

Before we can build a complex application that simulates workflows, we need to define the fundamental "language" we are speaking. Just as a sentence is made of nouns and verbs, a Petri net is built from three specific logical blocks.

We call these the **Petri Net Primitives**.

2.1 The Motivation: A Board Game Analogy

Imagine you are building a digital board game. Before you design the rules or the confusing strategies, you need to code the physical pieces:

1. **Circles (Places):** Where chips sit.
2. **Rectangles (Transitions):** Actions that move chips.
3. **Arrows (Arcs):** The paths showing where chips go.

In our application, we cannot just draw shapes on a screen. We need smart objects that know their own rules. A Circle needs to know how many chips it has. A Rectangle needs to know if it's allowed to move chips.

2.1.1 The Use Case: The Coffee Machine

Let's build a tiny piece of logic for a Coffee Machine.

- **Place:** `Water Tank` (Contains 1 unit of water).
- **Transition:** `Brew` (The action).
- **Goal:** Move the unit of water into the coffee pot.

This chapter walks you through the classes that make this possible.

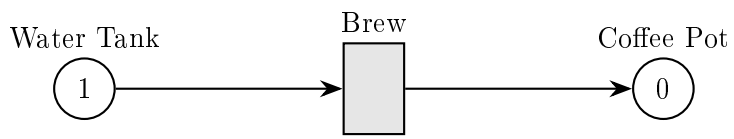

Figure 2.1: Coffee Machine Petri Net Example

2.2 The Key Concepts

2.2.1 The Place (`Place`)

Think of a **Place** as a bucket. It sits at a specific `x,y` coordinate on the screen. Its most important job is to hold **Tokens**.

- **Shape:** Always a Circle.
- **Data:** The number of tokens (e.g., `token: 5`).

2.2.2 The Transition (`Transition`)

Think of a **Transition** as an engine or a processor. It waits for resources to arrive, consumes them, and produces something new.

- **Shape:** Always a Rectangle (or Box).
- **Logic:** It checks to see if there are enough tokens in the connected Places to "fire" (execute).

2.2.3 The Arc (`Arc`)

The **Arc** is the pipe connecting them. It is strictly directional.

- **Shape:** A line with an arrow.
- **Logic:** It connects a `Node` (Place/Transition) to another `Node`.

2.3 Using the Primitives

Let's look at how we create these objects in code to solve our **Coffee Machine** use case.

2.3.1 Step 1: Create the Water Tank (Place)

We define a location and create a `Place` with 1 token.

```
import { Point } from './point';
import { Place } from './place';

// Define position (x=100, y=100)
const tankPos = new Point(100, 100);

// Create Place: 1 token, at position, ID='p1', Label='Water'
const waterTank = new Place(1, tankPos, 'p1', 'Water');
```

What happened? We now have a logic object representing a tank with 1 unit of "water" (token).

2.3.2 Step 2: Create the Brew Action (Transition)

Now we create the action that will use the water.

```
import { Transition } from './transition';

// Define position nearby (x=300, y=100)
const brewPos = new Point(300, 100);

// Create Transition
const brewAction = new Transition(brewPos, 't1', 'Brew');
```

2.3.3 Step 3: Connect them (Arc)

We need a pipe to move water to the brewer.

```
import { Arc } from './arc';

// Create an Arc from Tank -> Brew Action
// Weight 1 means it moves 1 token at a time
const pipe = new Arc(waterTank, brewAction, 1);

// Tell the transition about this incoming connection
pipe.appendSelfToTransition();
```

What happened? The `brewAction` now knows it is connected to the `waterTank`. Specifically, `waterTank` is a "Pre-Arc" (input) for the transition.

2.4 Under the Hood: Geometry & Logic

Before diving into the source code, let's understand a hidden complexity.

When we draw this on screen in the Main Visualizer (Chapter 4), we want the arrow to stop exactly at the edge of the circle or square. We don't want the arrow overlapping the shape, and we don't want a gap.

Each primitive knows how to calculate its **Intersection Point**.

2.4.1 Sequence Diagram: Drawing a Line

Here is what happens when the visualizer asks an Arc, "Where are your points?"

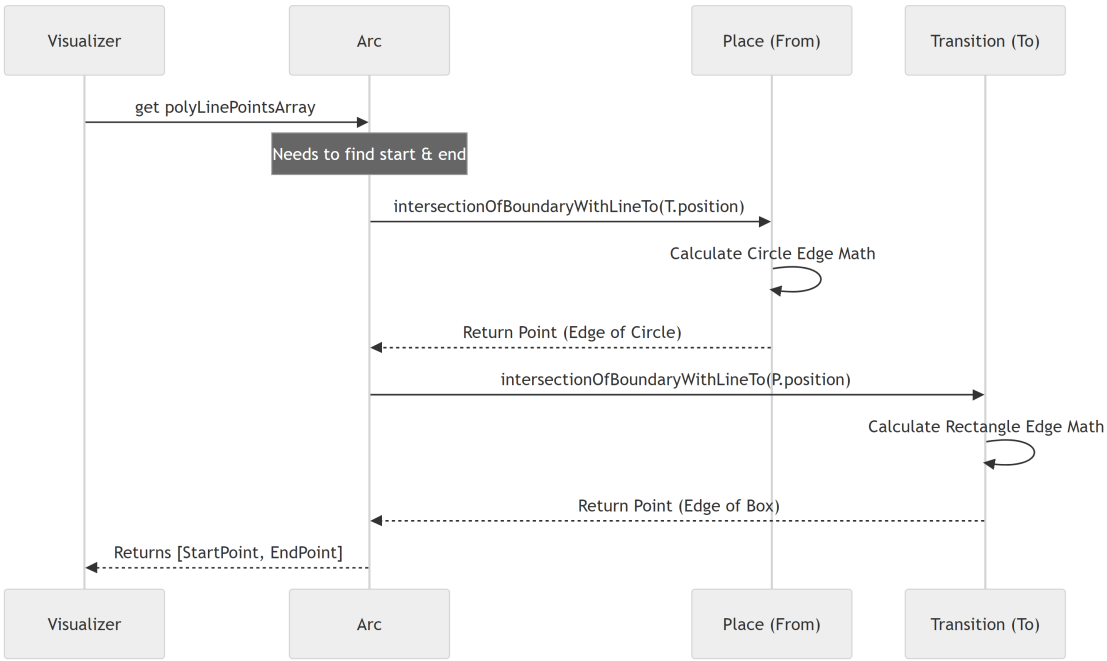

Figure 2.2: Sequence Diagram: Arc requests boundary intersection points

2.5 Deep Dive: The Code

Let's look at the actual files provided.

2.5.1 The `Place` Class (`place.ts`)

This class implements `Node`. Notice the `intersectionOfBoundaryWithLineTo` method. This is pure geometry.

```
export class Place implements Node {
    token: number;
    position: Point;
    // ... id and label properties

    // Calculates where a line hits the edge of this circle
    intersectionOfBoundaryWithLineTo(p: Point): Point {
        const r = radius; // Defined in constants
        const m = this.position; // Center of circle

        // ... Math calculation (Standard Sine/Cosine trig) ...

        // Returns the point exactly on the circle's edge
        return new Point(xIntersect, yIntersect);
    }
}
```

2.5.2 The `Transition` Class (`transition.ts`)

The transition is smarter. It has logic to check if it's "Active". A transition is active if all the places feeding into it have enough tokens.

```
export class Transition implements Node {
    // ... properties ...

    // Checks if we have enough tokens to fire
    get isActive(): boolean {
        for (let preArc of this.preArcs) {
            // Check if the source Place has enough tokens
            // Note: arc weights are stored as negative numbers for inputs
            if ((preArc.from as Place).token + preArc.weight < 0) {
                return false;
            }
        }
        return true; // All input places satisfied!
    }
}
```

2.5.3 The `Arc` Class (`arc.ts`)

The Arc acts as the bridge. Notice how the constructor automatically adjusts weights: negative if leaving a Place (removing tokens), positive if entering a Place (adding tokens).

```
export class Arc {
    from: Node;
    to: Node;
    weight: number;

    constructor(from: Node, to: Node, weight: number = 1) {
        this.from = from;
        this.to = to;

        // If coming FROM a Place, weight is negative (subtraction)
        if (from instanceof Place) {
            this.weight = -1 * Math.abs(weight);
        } else {
```

```
            this.weight = Math.abs(weight);
        }
    }
    // ...
}
```

Also, the Arc handles the geometry request we saw in the Sequence Diagram:

```
    get polyLinePointsArray(): Point[] {
        // ...

        // Ask the 'From' node where the line should start
        start = this.from.intersectionOfBoundaryWithLineTo(pForStartCalc);

        // Ask the 'To' node where the line should end
        end = this.to.intersectionOfBoundaryWithLineTo(pForEndCalc);

        return [start, ...this.anchors, end];
    }
```

2.6 Conclusion

You have now mastered the alphabet of our application!

- **Places** hold the state.
- **Transitions** hold the rules.
- **Arcs** connect them and handle the geometry of drawing lines.

However, having these objects floating around in memory isn't enough. We need a way to manage them, create them, delete them, and find them by ID. We need a "Manager".

In the next chapter, we will build the brain that holds all these pieces together.

Next Chapter: Central Data Store (DataService)

Chapter 3

Central Data Store (DataService)

In the previous chapter, Petri Net Primitives, we learned how to create the fundamental building blocks: Places, Transitions, and Arcs.

But right now, those objects are just ghost logic floating in the void. We have "bricks," but we don't have a "building" or a "construction site."

If you create a `Place` in one code file, how does the drawing engine know it exists? If you delete a `Transition` in the layout tool, how does the simulation engine know it's gone?

We need a single, shared brain. We call this the **DataService**.

3.1 The Motivation: The Shared Whiteboard

Imagine a team of engineers working in a room. To collaborate effectively, they use a large **Whiteboard** on the wall.

- The **Visualizer** looks at the board to see what to draw.
- The **Simulation** reads the board to move tokens around.
- The **Mouse Handler** erases things from the board when you click delete.

The `DataService` is that whiteboard. It is the **Single Source of Truth**.

3.1.1 The Use Case: "Connecting the Dots"

Let's stick with our coffee machine example. We have a `Water Tank` (Place) and a `Brew` (Transition).

Goal: We want to connect them with an Arc and ensure the entire application looks at the new connection instantly.

3.2 Key Concepts

3.2.1 The Storage Arrays

The DataService holds private lists (arrays) of your items.

- `_places`: A list of all circles.
- `_transitions`: A list of all rectangles.
- `_arcs`: A list of all arrows.

3.2.2 The "Town Crier" (Observables)

This is the most "magical" part of the service. In modern web development, simply changing a variable isn't enough; you have to tell the screen to update.

We use a concept called **RxJS Observables**. Think of it as a **Town Crier** or a **Group Chat**.

1. **Subject (`dataChangedSubject`):** This is the megaphone. When we change data, we shout into this.
2. **Observable (`dataChanged$`):** This is what the components listen to. When the megaphone shouts, everyone listening wakes up and refreshes their screen.

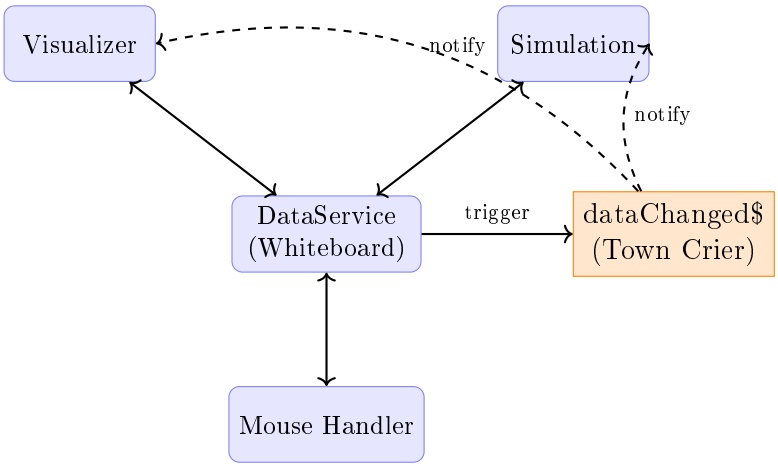

Figure 3.1: DataService as Single Source of Truth with Observable Pattern

3.3 Using the DataService

Let's solve our "Connecting the Dots" use case.

3.3.1 Step 1: Getting Access

In Angular, we don't say `new DataService()`. We ask the framework to give us the shared instance. This is called **Dependency Injection**.

```
// Inside a component
constructor(private dataService: DataService) {
   // Now 'this.dataService' is the shared whiteboard
}
```

3.3.2 Step 2: Adding Elements

We define our primitives (like in Chapter 2) and put them into the service's arrays.

```
// Create the objects
const tank = new Place(1, new Point(0,0), 'p1');
const brew = new Transition(new Point(100,0), 't1');

// Put them on the "Whiteboard"
this.dataService.places = [tank];
this.dataService.transitions = [brew];
```

3.3.3 Step 3: Connecting Them

The `DataService` has a helper method called `connectNodes`. This is smarter than doing it manually because it automatically creates the `Arc` and updates the internal logic.

```
// Connect Tank -> Brew
// The service creates a new Arc and saves it internally
this.dataService.connectNodes(tank, brew);
```

What happened? An Arc was created. The `brew` transition now knows `tank` is its input. But... the screen hasn't updated yet!

3.3.4 Step 4: The Notification

Usually, methods like `connectNodes` trigger the update automatically. But if you wanted to listen to updates in your component, you would do this:

```
// Listen for the Town Crier
this.dataService.dataChanged$.subscribe(() => {
    console.log("Something changed! I should redraw the screen.");
    this.redrawPetriNet();
});
```

3.4 Under the Hood: Internal Logic

How does the service keep everything essentially organized? Let's trace what happens when we delete something.

3.4.1 Sequence Diagram: Deleting a Place

Imagine the user selects a Place and hits "Delete".

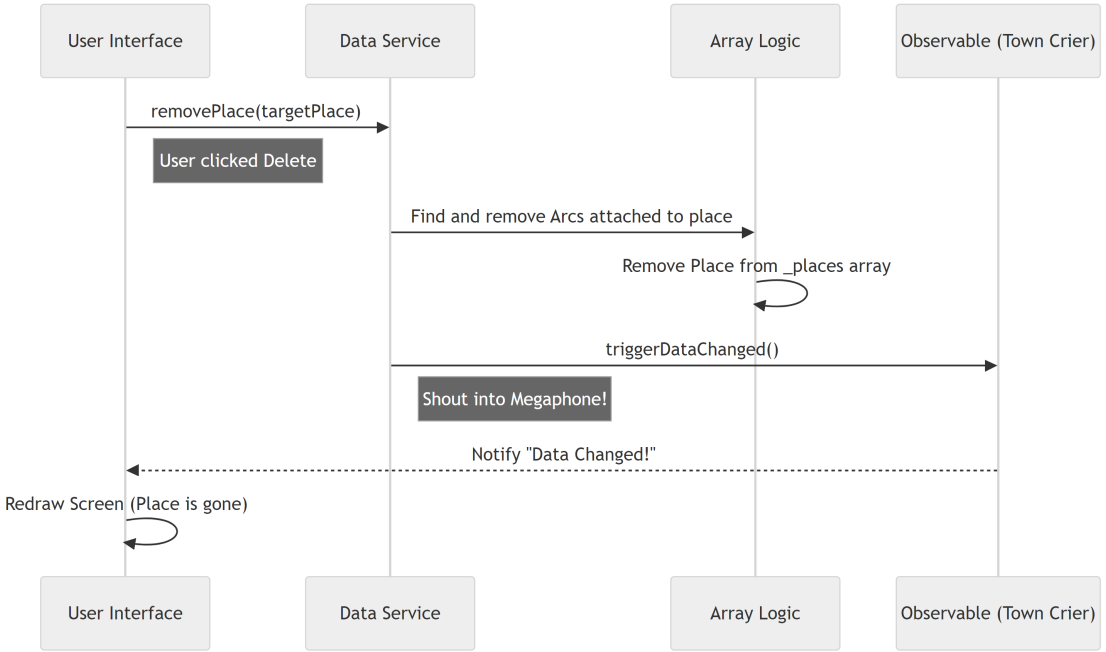

Figure 3.2: Sequence Diagram: Deleting a Place triggers cascading updates

3.5 Deep Dive: The Code

Let's look at `src/app/tr-services/data.service.ts` to see how this is built.

3.5.1 The Storage and Safety

The service keeps the data `private`. This prevents other parts of the app from accidentally wiping the array. We access data using **Getters**.

```
export class DataService {
    // Private storage - the "Real" data
    private _places: Place[] = [];
    private _transitions: Transition[] = [];
    private _arcs: Arc[] = [];

    // Public access - Read Only access
    getPlaces(): Place[] {
        return this._places;
    }
    // ... similar getters for transitions and arcs
}
```

3.5.2 The Smart Removal Logic

When removing a Place, we can't just delete the circle. We must also delete any arrows (arcs) connected to it, otherwise, we'd have arrows pointing to nothing (which crashes the app).

```
removePlace(deletablePlace: Place): Place[] {
    // 1. Find all arcs touching this place
    const deletableArcs = this._arcs.filter(
        (arc) => arc.from === deletablePlace || arc.to === deletablePlace,
    );

    // 2. Remove those arcs first
    deletableArcs.forEach((arc) => this.removeArc(arc));

    // 3. Remove the place itself
    this._places = this._places.filter((place) => place !== deletablePlace);

    // 4. Shout to the app that data changed
    this.triggerDataChanged();
    return this._places;
}
```

3.5.3 The "Trigger" Mechanism

This is the heart of the reactivity. Whenever you modify data (add, remove, connect), you call this method.

```
// The Town Crier Subject
private dataChangedSubject = new Subject<{fitContent: boolean}>();

// The Public Listener
public dataChanged$ = this.dataChangedSubject.asObservable();

public triggerDataChanged(fitContent: boolean = false): void {
    // Emit the event. fitContent tells the camera if it should zoom to fit.
    console.log('Triggering Update');
    this.dataChangedSubject.next({ fitContent });
}
```

3.6 Conclusion

The **DataService** is the backbone of our application. It ensures that no matter how many components we build—layout engines, simulators, or exporters—they all look at the exact same Petri net model.

- It holds the Arrays (`_places`, etc.).
- It protects data integrity (deleting Arcs when Places are deleted).
- It notifies the app when changes happen via `dataChanged$`.

Now that we have a place to store our data, we need a way to actually **see** it. In the next chapter, we will build the component that reads this data and draws it onto the screen.

Next Chapter: The Main Visualizer (PetriNetComponent)

Chapter 4

The Main Visualizer (PetriNetComponent)

In the previous chapter, Central Data Store (DataService), we built the application's "brain." We have a secure place to store Places, Transitions, and Arcs.

But currently, our application is invisible. The data exists in memory, but the user sees a blank screen.

We need a "Painter." We need a component that takes the raw lists of objects and renders them into beautiful, interactive shapes. This is the job of the **PetriNetComponent**.

4.1 The Motivation: The Artist Analogy

Imagine you are directing a play.

- **The Script (DataService):** Contains all the characters and instructions.
- **The Stage (PetriNetComponent):** This is where the visualization happens.

The Stage has two responsibilities:

1. **Rendering:** If the script says "Enter Hamlet," the stage must show an actor.
2. **Interaction:** If the audience throws a tomato (clicks a mouse), the stage must react.

4.1.1 The Use Case: "The Magic Canvas"

Goal: The user clicks on a blank white area. A circle (Place) appears.

This sounds simple, but it involves translating a pixel coordinate (Mouse X,Y) into a logical object, storing it, and then drawing it back onto the screen.

4.2 Key Concepts

4.2.1 SVG (Scalable Vector Graphics)

We don't draw with pixels (like MS Paint). We draw with math. We use an **SVG Canvas**.

- To draw a Place, we write `<circle cx="100" cy="100" r="30" />`.
- To draw a Transition, we write `<rect x="200" y="200" ... />`.

4.2.2 The Loop (*ngFor)

We do not manually write <circle> for every single place. We use Angular's specific HTML command *ngFor.

Think of this as a **Photocopier**. We give it one template (a circle) and a list of data (Places array). It photocopies the circle for every item in the list.

4.2.3 The Event Dispatcher

This component acts like a **Traffic Cop**. When you click the mouse, the Visualizer asks: "What tool are you holding?"

- If you hold the **"Add"** tool: It adds a Place.
- If you hold the **"Delete"** tool: It removes the item you clicked.

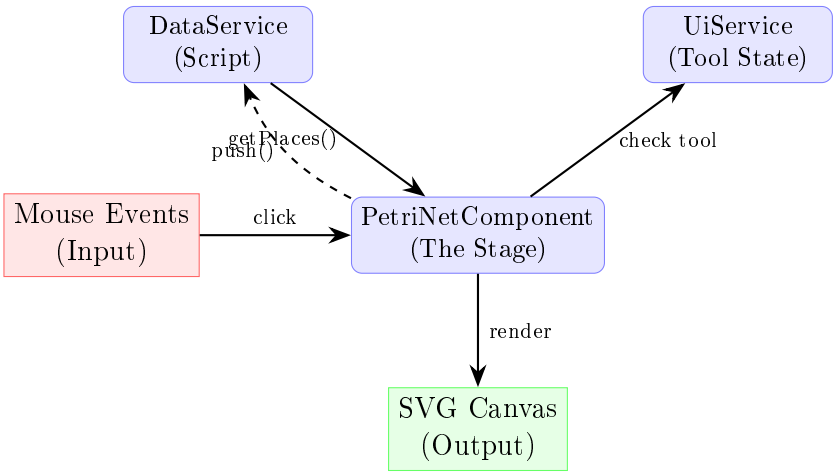

Figure 4.1: PetriNetComponent: The Stage between Data and Display

4.3 Using the Visualizer

Let's see how this component solves the "Magic Canvas" use case.

4.3.1 Step 1: The Template (HTML)

In petri-net.component.html, we define the canvas. We use the **DataService** to get the list of places.

```
<!-- The Canvas -->
<svg #drawingArea id="drawingArea" class="canvas">

    <!-- The Loop: Create a group <g> for every place -->
    <g *ngFor="let place of dataService.getPlaces()">

        <!-- Draw the circle using the place's position -->
        <circle
            [attr.cx]="place.position.x"
            [attr.cy]="place.position.y"
            class="place">
        </circle>

        <!-- Draw the number of tokens inside -->
        <text [attr.x]="place.position.x" [attr.y]="place.position.y">
```

```
            {{ place.token }}
        </text>
    </g>
</svg>
```

What happened? If the DataService has 3 places, Angular automatically draws 3 circles on the screen.

4.3.2 Step 2: Reacting to Changes

The component needs to know when the DataService changes so it can refresh the drawing. We do this in the initialization code (`ngOnInit`).

```
// petri-net.component.ts
ngOnInit(): void {
    // Subscribe to the "Town Crier" from the DataService
    this.dataService.dataChanged$.subscribe((data) => {

        // When data changes, maybe fit the view (zoom)
        if (data?.fitContent) {
            this.fitContentToView();
        }
        // Angular automatically re-renders the *ngFor loops
    });
}
```

4.3.3 Step 3: Handling Clicks

When the user clicks the canvas, we need to decide what to do.

```
// Inside petri-net.component.ts
dispatchSVGClick(event: MouseEvent, drawingArea: HTMLElement) {
    event.preventDefault();

    // Check what button is currently active in the toolbar
    if (this.uiService.button === ButtonState.Place) {
        // If "Place" button is active, create a place!
        this.addPlace(event, drawingArea);
    }
}
```

4.4 Under the Hood: The Interaction Loop

Ideally, drawing should be instant. But logically, it follows a strict cycle.

1. User clicks Canvas.
2. Visualizer calculates where (X, Y).
3. Visualizer tells **DataService** “Add a Place here.”
4. **DataService** updates the array.
5. **DataService** shouts “Data Changed!”
6. Visualizer hears the shout and updates the HTML.

4.4.1 Sequence Diagram: Creating a Place

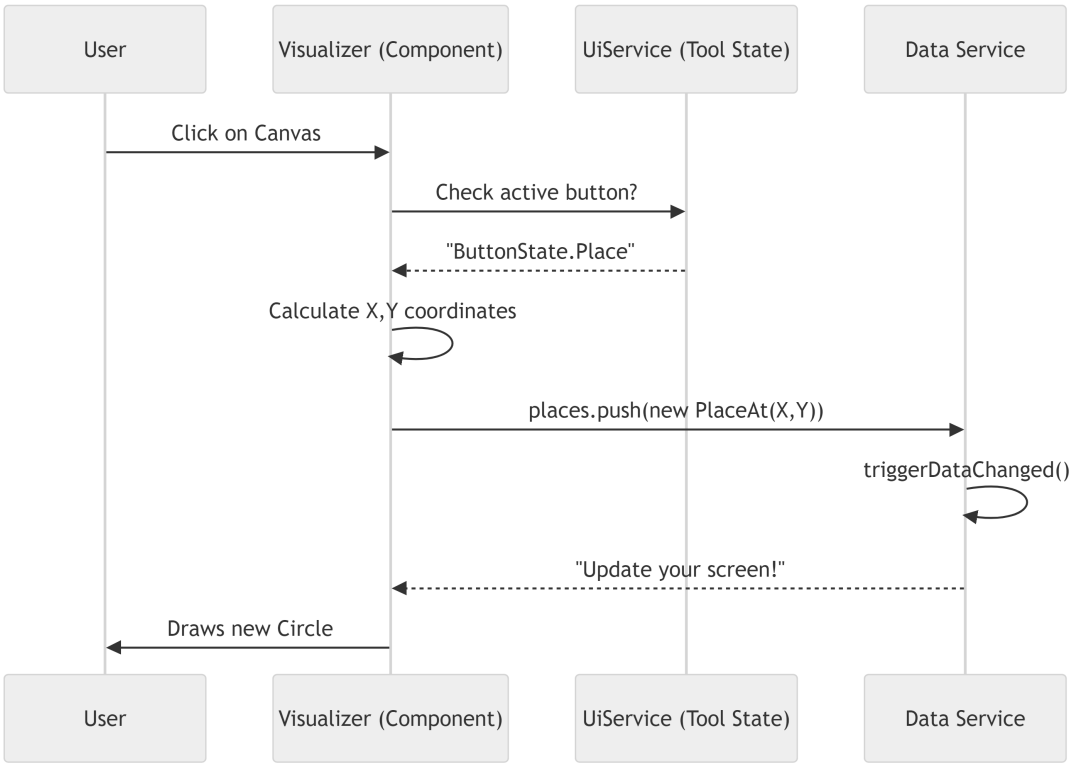

Figure 4.2: Sequence Diagram: Creating a Place via Click

4.5 Deep Dive: Key Code Blocks

Let's look at the actual implementation in `petri-net.component.ts`.

4.5.1 Translating Mouse Coordinates

A mouse click gives coordinates relative to the *window* (e.g., top-left of your browser). We need coordinates relative to the *SVG canvas*.

```
createPlace(event: MouseEvent, drawingArea: HTMLElement): Place {
    // Helper service to do the math (Window X/Y -> SVG X/Y)
    const point = this.svgCoordinatesService.getRelativeEventCoords(
        event,
        drawingArea,
    );

    // Create new Place with 0 tokens at that point
    return new Place(0, point, this.getPlaceId());
}
```

4.5.2 Styling States (`ngClass`)

We don't just draw white circles. Sometimes circles are red (error), green (simulation), or have shadows (editable). We use `[ngClass]` to swap CSS classes dynamically.

```
<circle
```

```
    [ngClass]="{
        'place': true,                          <!-- Always applies -->
        'editable': isPlaceEditable(place),     <!-- Only if editable -->
        'active': place === selectedPlace       <!-- Only if selected -->
    }"
    ...
>
</circle>
```

4.5.3 Displaying Tokens

A Petri net place isn't just a circle; it holds tokens. In the HTML, we overlay text centered on the circle.

```
<text class="token-label"
      [attr.x]="place.position.x"
      [attr.y]="place.position.y"
      text-anchor="middle"
      dominant-baseline="central">

    <!-- Only show number if tokens > 0 -->
    {{ place.token > 0 ? place.token : "" }}
</text>
```

4.5.4 Running the Animation (Simulation)

This component is also responsible for showing the "movie" when we simulate the net. It uses a timer to update the screen step-by-step.

```
private animateNextStep(): void {
    // 1. Get the next step from the script
    const currentStep = this.uiService.getCurrentSimulationStep();

    // 2. Display the state on screen
    this.displayStateForStep(currentStep);

    // 3. Wait a bit, then do it again (Recursive Loop)
    this.animationTimer = setTimeout(() => {
        this.animateNextStep();
    }, 1000); // 1-second delay
}
```

4.6 Conclusion

The **PetriNetComponent** is the window into our application. It doesn't "own" the data (the DataService does that), but it makes the data usable.

It handles the magic triangle of:

1. **Events:** Listening to your mouse.
2. **Logic:** Asking "What tool are we using?"
3. **Rendering:** Drawing SVG shapes based on data arrays.

Now that we can verify our internal data structures by seeing them on screen, we run into a new problem. How do we save this diagram? Or how do we load a standard Petri net file from the internet?

We need a translator.

Next Chapter: PNML/XML Translator (PnmlService)

Chapter 5

PNML/XML Translator (PnmlService)

In the previous chapter, The Main Visualizer (PetriNetComponent), we learned how to draw our Petri net on the screen. We can click to add Places and Transitions, and everything looks great.

But we have a major problem: **Amnesia**.

If you refresh your browser, your drawing disappears. If you want to send your Petri net to a friend, you can't. The data only lives in your computer's temporary memory (RAM).

We need a way to save our work to a file. We use the **PnmlService** to solve this.

5.1 The Motivation: The Universal Translator

Computer programs speak different languages.

- **Our App:** Speaks "JavaScript Objects" (e.g., `new Place(...)`).
- **The File System:** Speaks "Text/String" (e.g., `<place id="p1">`).

The standard language for saving Petri nets is called **PNML** (Petri Net Markup Language). It is based on **XML** (similar to HTML).

We need a **Translator**:

1. **Import (Read):** Reads an XML file and creates JavaScript objects.
2. **Export (Write):** Takes JavaScript objects and writes an XML file.

5.1.1 The Use Case: Saving and Loading

- **Goal:** You draw a coffee machine workflow and click "Download."
- **Result:** A file named `petri-net.pnml` is downloaded to your computer.
- **Later:** You upload that file, and the app reconstructs the drawing exactly as it was.

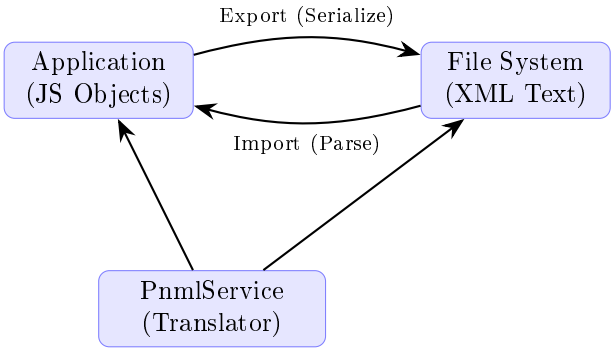

Figure 5.1: PnmlService as Universal Translator between App and Files

5.2 Key Concepts

5.2.1 PNML (XML)

PNML is just text wrapped in tags. It describes the net's structure. Here is what a single Place looks like in PNML:

```
<place id="p1">
    <name> <text>Water Tank</text> </name>
    <graphics>
        <position x="100" y="100"/>
    </graphics>
</place>
```

5.2.2 Parsing (Deserialization)

Parsing is the act of reading that text and extracting meaning.

- The translator looks for `<position x="100">`.
- It converts `"100"` (text) into `100` (number).
- It runs `new Place(...)`.

5.2.3 Serialization

Serialization is the reverse.

- It looks at a `Place` object.
- It gives it a string template: `<place id="${place.id}">...`.
- It combines them all into one long text string.

5.3 Using the Service

Let's see how we use this service to save and load data.

5.3.1 Exporting (Downloading)

The `PnmlService` makes this incredibly easy. It grabs the data directly from the Central Data Store (DataService).

```
// Inside a toolbar component
exportNet() {
    // 1. Ask the service to write existing data to a file
    this.pnmlService.writePNML();
}
```

What happens? The browser will immediately prompt you to download a file named `petri-net-with-love.pnml`.

5.3.2 Importing (Uploading)

When a user uploads a file, we read the text inside it and pass it to the service.

```
// Assume 'fileContent' is the string read from the uploaded file
importNet(fileContent: string) {

    // 1. Pass the XML string to the parser
    this.pnmlService.parse(fileContent);

    // Note: The service automatically updates the DataService!
}
```

What happens?

1. The service reads the file.
2. It deletes the current net.
3. It creates the new Places/Transitions/Arcs.
4. It tells the application to zoom in (Fit to View).

5.4 Under the Hood: The Parsing Flow

How does a text file become a running application?

1. **Dependencies:** We use a library called `xml2js`. It converts the XML text into a raw, messy JSON object.
2. **Generic Processing:** The `PnmlService` iterates through that messy object looking for keywords like "place" or "arc".
3. **Instantiation:** For every match, it creates a real Class instance (`new Place()`).
4. **Auto-Layout:** If the file doesn't have coordinates (X, Y), the service calls the Graph Layout Architect to calculate them automatically.

5.4.1 Sequence Diagram: Import Process

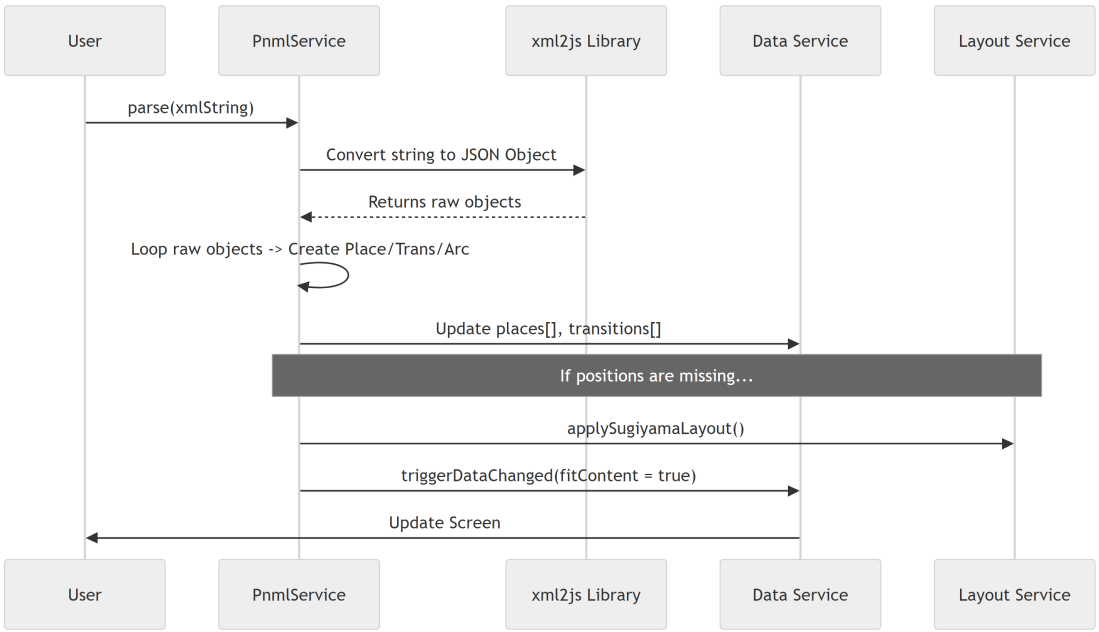

Figure 5.2: Sequence Diagram: PNML Import Process

5.5 Deep Dive: The Code

Let's look at `src/app/tr-services/pnml.service.ts`.

5.5.1 Generating XML (The Template)

When generating the text file, we map every object to a string. Notice how we manually construct the XML tags.

```
// Helper to convert a Place object to an XML string
getPlaceString(place: Place): string {
    return `      <place id="${place.id}">
    <name>
      <text>${place.label}</text>
    </name>
    <graphics>
      <position x="${place.position.x}" y="${place.position.y}"/>
    </graphics>
  </place>`;
}
```

5.5.2 The Main Parse Function

The `parse` method is the controller. It delegates the hard work to specific functions.

```
parse(xmlString: string) {
    // 1. Convert XML string to a raw Javascript Object
    const result = xml2js(xmlString) as PnmlPetriNet;

    // 2. Extract specific lists from the raw object
    // (Implementation details hidden: filtering for 'place', 'transition')
```

```
    // 3. Convert raw data into Application Class instances
    const places = this.parsePnmlPlaces(pnmlPlaces);
    const transitions = this.parsePnmlTransitions(pnmlTransitions);

    // 4. Update the Single Source of Truth
    this.dataService.places = places;
    this.dataService.transitions = transitions;

    // 5. Update the UI
    this.dataService.triggerDataChanged(true);
}
```

5.5.3 Parsing Specific Elements

Here is how we translate the raw data into a `Place`. We have to be careful—sometimes attributes are missing (like coordinates), so we set defaults.

```
private parsePnmlPlaces(list: Array<PnmlElement>): Place[] {
    const places: Place[] = [];

    list.forEach((pnmlPlace) => {
        // Extract ID and Name
        const id = pnmlPlace.attributes.id;

        // Extract Position (Logic simplified for readability)
        // If x/y exist, use them. If not, use (0,0).
        let point = new Point(attr.x, attr.y);

        // Create the real object
        places.push(new Place(tokens, point, id, name));
    });
    return places;
}
```

5.5.4 Handling Arcs (The Tricky Part)

Arcs are harder because they connect two things. In XML, an Arc is just: `source="p1" target="t1"`. In our code, an `Arc` needs references to the actual `Place` and `Transition` objects, not just their names.

```
private parsePnmlArcs(list, places, transitions): Arc[] {
    list.forEach((pnmlArc) => {
        // 1. Get the string IDs
        const sId = pnmlArc.attributes.source;
        const tId = pnmlArc.attributes.target;

        // 2. Find the actual objects in our memory
        const sourceNode = this.retrieveNode(places, transitions, sId);
        const targetNode = this.retrieveNode(places, transitions, tId);

        // 3. Create the connection
        if (sourceNode && targetNode) {
            const arc = new Arc(sourceNode, targetNode, weight);
            arcs.push(arc);
        }
    });
    return arcs;
}
```

5.6 Conclusion

The **PnmlService** is our bridge to the outside world.

- It allows users to save their work (Serialization).
- It allows the app to load existing nets (Parsing).
- It links the dumb text of a file to the smart objects in our **DataService**.

However, we mentioned a specific edge case: **What happens if the loaded file has no X/Y coordinates?**

Currently, the PnmlService just puts them at `(0,0)`, creating a jumbled pile of elements on top of each other.

We need an automatic architect to organize that pile into a readable graph.

Next Chapter: Graph Layout Architect (Sugiyama/Spring Embedder)

Chapter 6

Graph Layout Architect (Sugiyama/Spring Embedder)

In the previous chapter, PNML/XML Translator (PnmlService), we learned how to import Petri net files.

However, we ended on a cliffhanger. Sometimes, imported files contain only the *logic* (connections) but no *geometry* (positions). If a file doesn't say where the nodes go, the `PnmlService` piles them all at x=0, y=0.

This results in a "Tangled Mess"—a black blob of overlapping circles and rectangles.

We need an **Interior Designer**. We need an algorithm to automatically look at the connections and decide the best, most readable arrangement for the furniture. This is the job of the **Graph Layout Architect**.

6.1 The Motivation: The "Tangled Mess"

Imagine you just moved into a new house (imported a file). The movers dumped every single chair, table, and lamp into the exact center of the living room. You can't reach the fridge because the sofa is on top of it.

6.1.1 The Use Case: Auto-Layout

1. **Input:** A Petri net where every Place and Transition is at `(0,0)`.
2. **Action:** The user presses the "Auto Layout" button.
3. **Result:** The graph untangles itself. The nodes spread out, arrows point in a logical direction, and the diagram becomes readable.

We provide two different "Interior Designers" for two different tastes:

1. **Sugiyama:** Strict, organized, hierarchical (Top-to-Bottom).
2. **Spring Embedder:** Organic, physics-based (Explodes outward).

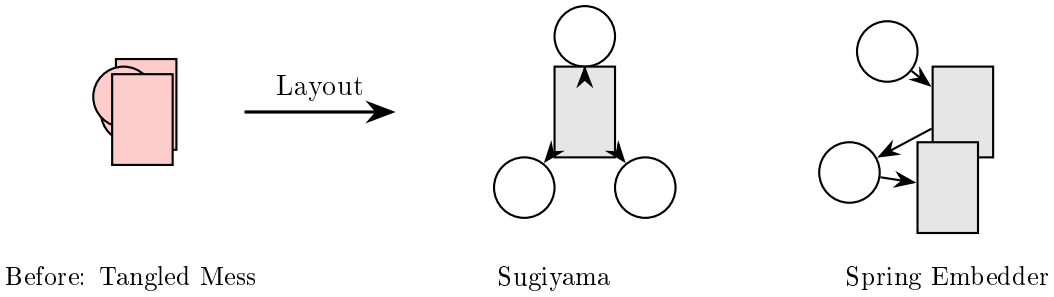

Figure 6.1: Layout transforms tangled nodes into readable graphs

6.2 Key Concepts

6.2.1 The Spring Embedder (Physics)

This approach treats the diagram like a physical system in space.

- **Magnets (Repulsion):** Every node (Place/Transition) hates every other node. They try to push each other away.
- **Springs (Attraction):** If two nodes are connected by an Arc, they are tied together with an elastic spring. They try to pull close.

The algorithm runs a simulation. The nodes push and pull until they find a comfortable balance (Equilibrium).

6.2.2 The Sugiyama (Hierarchy)

This approach treats the diagram like a corporate organizational chart or a waterfall.

- **Layers:** It organizes nodes into ranks. Input moves to Process, Process moves to Output.
- **Crossings:** It specifically tries to minimize the number of times arrows cross over each other.

6.3 Using the Layout Services

These services are tools you can call whenever the graph gets messy.

6.3.1 Using Spring Embedder

The `LayoutSpringEmbedderService` does the heavy lifting. You just need to tell it to start.

```
// Inside a component or toolbar
layoutWithPhysics() {
    // 1. Run the physics simulation
    // This moves the nodes in the DataService directly
    this.springEmbedderService.layoutSpringEmbedder();
}
```

6.3.2 Using Sugiyama

The `LayoutSugiyamaService` works similarly but is instantaneous (it calculates math rather than running a simulation loop).

```
layoutHierarchical() {
    // 1. Calculate layers and positions
    this.sugiyamaService.applySugiyamaLayout();

    // 2. Tell the visualizer to update
    this.dataService.triggerDataChanged();
}
```

6.4 Under the Hood: Spring Embedder Logic

Let's look at how the "Physics" engine works. It uses a loop that runs hundreds of times (iterations).

6.4.1 Sequence Diagram: The Physics Loop

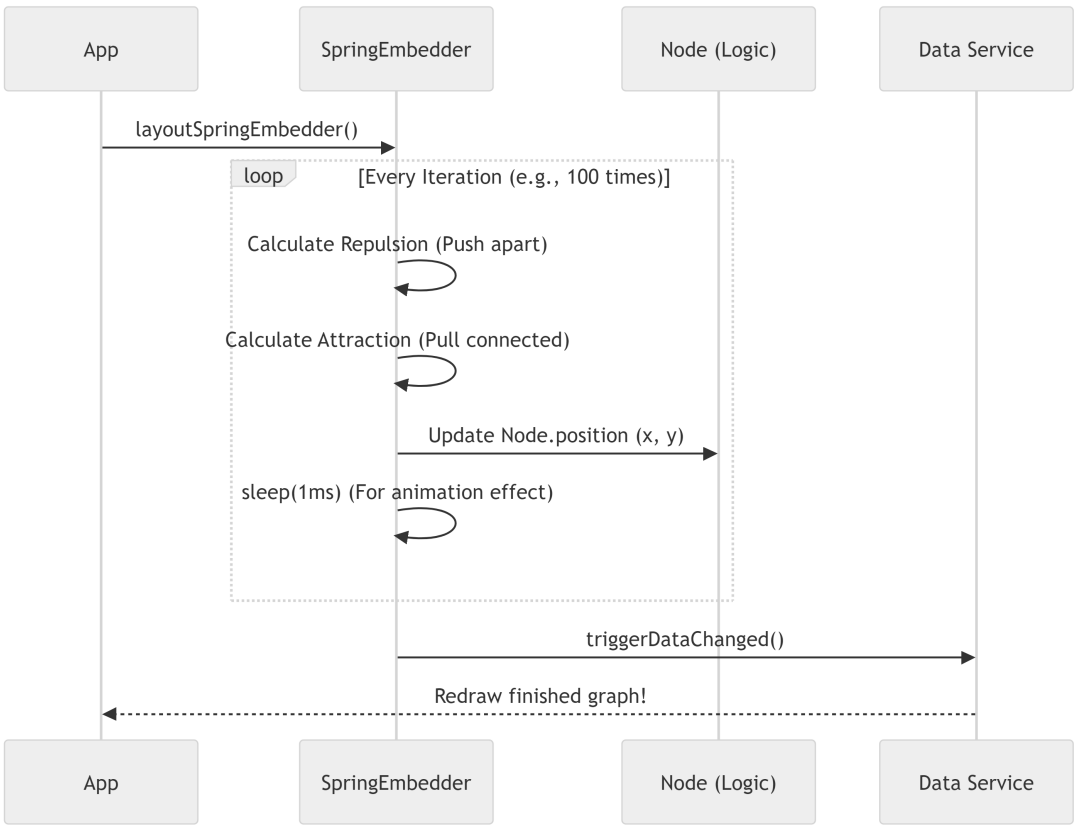

Figure 6.2: Sequence Diagram: Spring Embedder Physics Loop

6.4.2 Deep Dive: Spring Embedder Code

Open `src/app/tr-services/layout-spring-embedder.service.ts`.

The Main Loop

We run a `while` loop until the nodes stop moving significantly (stability) or we hit a limit.

```
async layoutSpringEmbedder() {
    let iterations = 1;

    // Keep running until max iterations or force is tiny
    while (iterations <= this.maxIterations) {

        // 1. Calculate where everyone wants to move
        // (Implementation details hidden in helper method)
        this.calculateAndApplyForces(nodes);

        // 2. Wait 1ms so user can see the movement (Animated)
        await this.sleep(1);

        iterations++;
    }
}
```

The Physics Math

We calculate two forces. Notice how simple the concept is when broken down:

```
// 1. Repulsion: "Get away from me!"
// Applied between ANY two nodes
const repulsionForce = this.calculateRepulsionForce(nodeA, nodeB);

// 2. Springs: "Come back!"
// Applied only between CONNECTED nodes (Arcs)
const springForce = this.calculateSpringForce(nodeA, connectedNodeB);

// Final move is the combination of both
totalForce = repulsionForce + springForce;
```

6.5 Under the Hood: Sugiyama Logic

The Sugiyama algorithm is much more complex, so we split it into four distinct steps. It is a **Pipeline**.

6.5.1 The 4-Step Pipeline

1. **Cycle Removal:** You can't make a perfect "waterfall" hierarchy if water flows back up (cycles). We temporarily reverse those arrows.
2. **Layer Assignment:** Assign every node a "Rank" (Row 1, Row 2, Row 3).
3. **Vertex Ordering:** Shuffle nodes within their row to stop arrows from crossing like shoelaces.
4. **Coordinate Assignment:** Finally, turn "Row 1, Position 2" into "x=100, y=50".

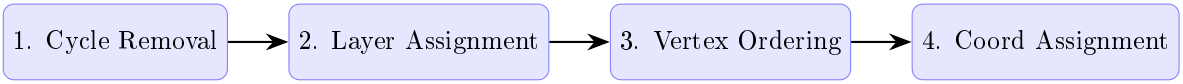

Figure 6.3: Sugiyama Algorithm: 4-Step Pipeline

6.5.2 Deep Dive: Sugiyama Code

Open `src/app/tr-services/layout-sugiyama.service.ts`.

Notice how the `applySugiyamaLayout` method acts as a manager delegating work to 4 sub-services. This is great software design!

```
applySugiyamaLayout() {
    // Step 1: Break infinite loops
    const cycleService = new CycleRemovalService(nodes, arcs);
    cycleService.removeCycles();

    // Step 2: Assign rows
    const layerService = new LayerAssignmentService(nodes, arcs);
    const layers = layerService.assignLayers();

    // Step 3: Shuffle to untangle lines
    const orderingService = new VertexOrderingService(layers, nodes, arcs);
    orderingService.orderVertices();

    // Step 4: Finalize X/Y coordinates
    const coordService = new CoordinateAssignmentService(layers, arcs, nodes);
    coordService.assignCoordinates();
}
```

6.5.3 Handling "Dummy Nodes"

Sometimes an arrow is very long, jumping from Layer 1 to Layer 5. This confuses the algorithm. The `VertexOrderingService` inserts invisible "Dummy Nodes" in layers 2, 3, and 4 to guide the arrow politely through the ranks.

```
// Inside vertex-ordering.service.ts
private insertDummyNodes() {
    // If an arrow skips a layer...
    if (layerDiff > 1) {
        // Create a fake node to fill the gap
        const dummy = new DummyNode();

        // Split the long arrow into two short ones
        this.splitArc(longArc, dummy);
    }
}
```

Why do this? It forces the straight lines to bend around obstacles, making the graph look much cleaner.

6.6 Conclusion

We now have a powerful set of tools to organize our Petri net.

- **Spring Embedder** is fun to watch and great for untangling clumps.
- **Sugiyama** is professional and great for structured workflows.

Both services modify the `position` of the nodes inside the Central Data Store (DataService).

Now the graph looks beautiful. But wait—what if the algorithm put a node slightly too far to the left? As a user, I want to grab it and move it myself. I want to rename it. I want to delete it.

We need to handle user interaction.

Next Chapter: Interaction Handler (EditMoveElementsService)

Chapter 7

Interaction Handler (EditMoveElementsService)

In the previous chapter, Graph Layout Architect (Sugiyama/Spring Embedder), we used powerful algorithms to automatically organize our Petri net.

The result is usually good, but rarely perfect. Maybe the algorithm placed a specific transition too close to a label, or perhaps you just prefer a specific layout for aesthetic reasons.

You need to reach into the screen and grab the objects. You need a "Hand."

This is the job of the **EditMoveElementsService**.

7.1 The Motivation: Physics of the Mouse

The browser gives us raw events: "The mouse is at pixel (500, 300)." Our objects live in a coordinate system: "Place P1 is at x=10, y=10."

If we just set `P1.x = 500`, the object would teleport instantly under your mouse cursor, which feels jarring and unnatural.

7.1.1 The Use Case: "The Fine Adjustment"

Goal: The user clicks on a Transition and drags it 10 pixels to the right.

Requirement:

1. Verify the user is allowed to move things (using the **Move Tool**).
2. Also move any arrows (Arcs) connected to that Transition so the lines don't break.

7.2 Key Concepts

7.2.1 The Drag Cycle

Every logic involving dragging follows this 3-step cycle:

1. **Initialize (`mousedown`):** "I have grabbed object X. I am starting at position Y."
2. **Update (`mousemove`):** "I have moved 2 pixels. Update object X by +2 pixels."
3. **Finalize (`mouseup`):** "I have let go. Forget object X."

Figure 7.1: The 3-Step Drag Cycle

7.2.2 The Delta (Δ)

We rarely set absolute coordinates during a drag. Instead, we calculate the **Difference** (Delta) between the last frame and the current frame.

- Frame 1: Mouse at 100.
- Frame 2: Mouse at 105.
- **Delta:** +5.
- **Action:** Add 5 to the Object's position.

7.2.3 Anchors (The Knee Joints)

An Arc is a line. But sometimes, you want a bent line. An **Anchor** is a point in the middle of a line. Imagine it like a knee joint. We can grab this joint and move it to bend the pipe around obstacles.

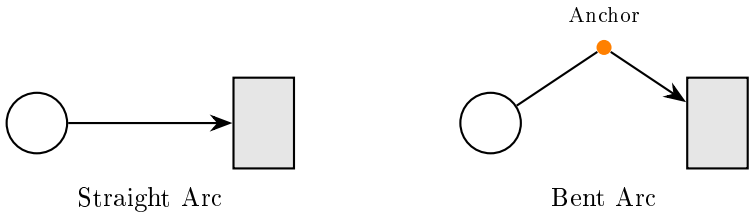

Figure 7.2: Anchors allow bending lines around obstacles

7.3 Using the Service

This service is primarily used by the Main Visualizer (PetriNetComponent) to react to mouse events.

7.3.1 Step 1: Grabbing an Object

When the user clicks, we tell the service what they clicked on.

```
// Inside the visualizer component (on mousedown)
onNodeDown(event: MouseEvent, node: Node) {
    if (this.uiService.button === ButtonState.Move) {
        // Prepare the service to move this specific node
        this.moveService.initializeNodeMove(event, node);
    }
}
```

7.3.2 Step 2: Dragging the Mouse

As the mouse moves, we ask the service to calculate the math.

```
// Inside the visualizer component (on mousemove)
onMouseMove(event: MouseEvent) {
    // This function automatically calculates the Delta
    // and updates the coordinates in the DataService
    this.moveService.moveNodeByMousePositionChange(event);
}
```

7.3.3 Step 3: Letting Go

When the user releases the mouse button.

```
// Inside the visualizer component (on mouseup)
onMouseUp() {
    // Clear the memory so we don't keep moving things
    this.moveService.finalizeMove();
}
```

7.4 Under the Hood: The Movement Logic

What actually happens when you drag a node? It's not just the node that moves; the attached lines must stretch or shrink.

7.4.1 Sequence Diagram: Dragging a Node

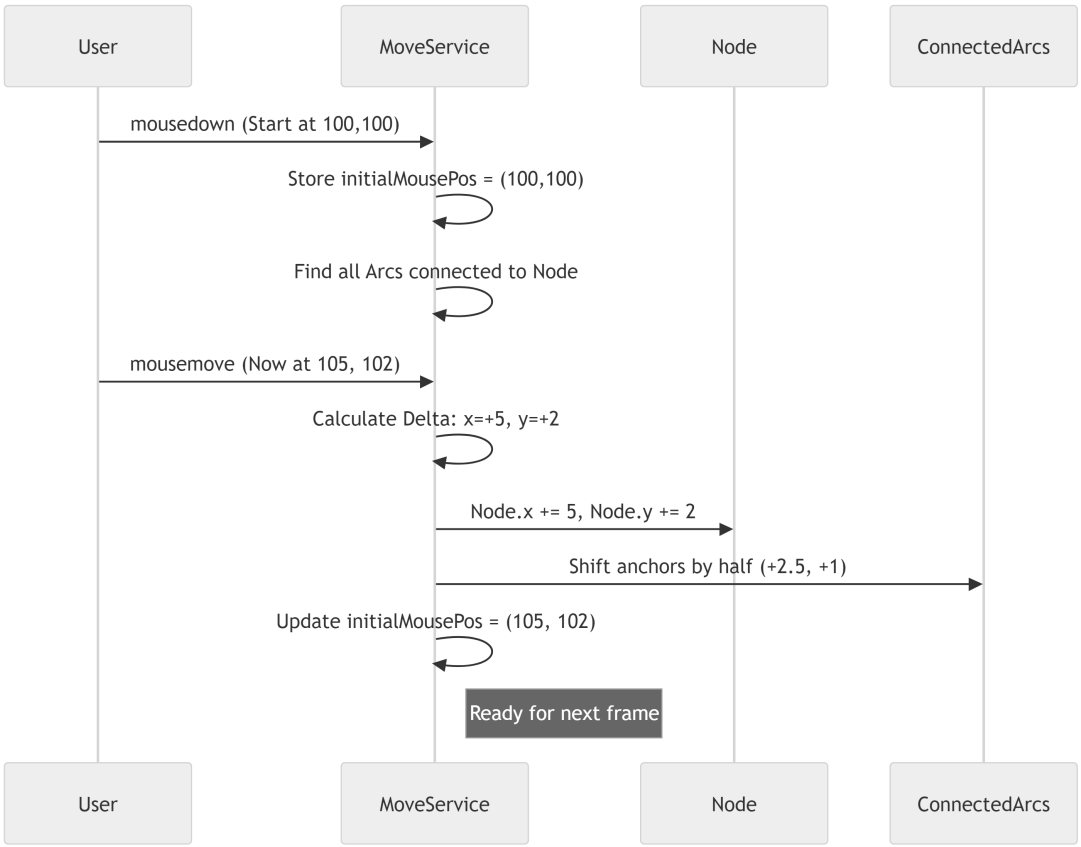

Figure 7.3: Sequence Diagram: Dragging a Node with Connected Arcs

7.5 Deep Dive: The Code

Let's look at `src/app/tr-services/edit-move-elements.service.ts`.

7.5.1 Initialization

When we start moving a node, we do a bit of setup work. We look through the Central Data Store (DataService) to find connected Arcs.

Why? If we didn't do this, dragging a Place would leave the arrows pointing at empty space (broken lines).

```
initializeNodeMove(event: MouseEvent, node: Node) {
    this.node = node;

    // Pre-calculate which arcs are attached to this node
    // This saves performance during the rapid drag loop
    this.dataService.getArcs().forEach((arc) => {
        if (arc.from === node || arc.to === node) {
            this.nodeArcs.push(arc);
        }
    });

    // Remember where we started
    this.initialMousePos.x = event.clientX;
    this.initialMousePos.y = event.clientY;
}
```

7.5.2 The Move Loop (The Physics)

This function is called dozens of times per second. It keeps the "physics" feeling responsive.

```
moveNodeByMousePositionChange(event: MouseEvent) {
    if (this.node) {
        // 1. Calculate how far the mouse has moved
        const deltaX = event.clientX - this.initialMousePos.x;
        const deltaY = event.clientY - this.initialMousePos.y;

        // 2. Apply that movement to the node
        this.node.position.x += deltaX;
        this.node.position.y += deltaY;

        // 3. Move connected lines slightly so they follow nicely
        this.moveConnectedAnchors(deltaX, deltaY);

        // 4. Reset tracker for the next frame
        this.initialMousePos.x = event.clientX;
        this.initialMousePos.y = event.clientY;
    }
}
```

7.5.3 Panning the Canvas

Sometimes you aren't grabbing a node; you are grabbing the background to move the whole map. This loops through **everything** in the DataService.

```
movePetrinetPositionByMousePositionChange(event: MouseEvent) {
    // Calculate Delta
    const deltaX = event.clientX - this.initialMousePos.x;

    // Apply Delta to EVERY Place and Transition
    [...this.dataService.getPlaces(), ...this.dataService.getTransitions()]
        .forEach((node) => {
            node.position.x += deltaX;
            // ...
        });

    // Also move every Arc Anchor
    // ...
}
```

7.5.4 Bending Lines (Inserting Anchors)

This is a cool feature. If a line is straight, you can click the middle of it to create a "joint." The service uses `SvgCoordinatesService` to translate the screen click into the chart's math coordinates.

```
insertAnchorIntoLineSegmentStart(event, arc, lineSegment, drawingArea) {
    // 1. Calculate where exactly on the map you clicked
    const anchor = this.svgCoordinatesService.getRelativeEventCoords(
        event,
        drawingArea
    );

    // 2. Add this point to the Arc's list of joints
    arc.anchors.push(anchor);

    // 3. Immediately switch the tool to "Move"
    // This lets the user drag the new joint instantly
    this.uiService.button = ButtonState.Move;
    this.initializeAnchorMove(event, anchor);
}
```

7.6 Conclusion

The **EditMoveElementsService** brings the static picture to life.

- It handles the **Delta Math** (Current Position − Old Position).
- It performs **Cascading Updates** (Moving a node also adjusts its connected Arcs).
- It allows for **Complex Interactions** like bending lines by inserting anchors.

We can now organize our graph manually by dragging elements around. But what if the graph is huge? What if we need to see the "Big Picture" or zoom in on a tiny detail?

Moving the elements isn't enough; we need to move the *camera*.

Next Chapter: Camera Controller (ZoomService)

Chapter 8

Camera Controller (ZoomService)

In the previous chapter, Interaction Handler (EditMoveElementsService), we learned how to grab specific nodes and move them around. We became the "Hand" that arranges the furniture.

But what if the room is too big? What if you stick a node in the far-right corner, and now you have to scroll for five minutes to find it? Or what if you import a massive Petri net that is 50 times wider than your screen?

Moving the *objects* isn't enough. We need to move the *camera*. We need to control the **Viewport**.

This is the job of the **ZoomService**.

8.1 The Motivation: Looking Through the Lens

Imagine holding a physical map.

- **Panning (Offset):** You slide the map left, right, up, or down on the table to see a different city.
- **Zooming (Scale):** You bring your face closer to the map to see street names, or pull back to see the whole country.

In our code, the **SVG Canvas** is that map. The `ZoomService` calculates exactly how much to "slide" (translate) and "magnify" (scale) that canvas so the user sees exactly what they want.

8.1.1 The Use Case: "Fit to View"

This is the most critical feature in any diagramming tool.

1. **Scenario:** User imports a file with 500 nodes.
2. **Problem:** The nodes are scattered over a huge area (e.g., from x=0 to x=5000). The user sees a white screen because the camera is looking at an empty spot.
3. **Solution:** The user clicks **"Fit Content."**
4. **Result:** The service calculates the perfect zoom level (Scale) and position (Offset) to fit every single node perfectly onto the screen.

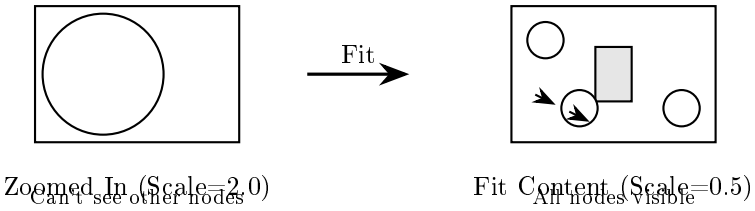

Figure 8.1: Fit Content adjusts Scale and Offset to show all nodes

8.2 Key Concepts

8.2.1 The State Variables

The camera is defined by just two sets of numbers:

1. **Scale (Zoom Level):** A multiplier.
   - `1.0` = Actual size (100%).
   - `0.5` = Half size (Zoomed out).
   - `2.0` = Double size (Zoomed in).
2. **Offset (Pan Position):** An `x, y` coordinate. This tells us how many pixels we shifted the canvas from the top-left corner.

8.2.2 The CSS Transform

Ultimately, this service produces a single string of text that the browser understands. It looks like this: `translate(100px, 50px) scale(1.5)`. This tells the browser: "Move everything right 100px, down 50px, and make it 50% bigger."

8.2.3 The "Pin" (Zoom Point)

When you use a map app (like Google Maps) and scroll your mouse wheel, the map zooms *towards your mouse cursor*. It feels like sticking a pin in the map at your mouse position; everything expands around that pin. This requires some clever math.

8.3 Under the Hood: The Flow

How does the application know to update the screen? Just like the Central Data Store (DataService), the `ZoomService` triggers updates that other components listen to.

8.3.1 Sequence Diagram: Clicking "Zoom In"

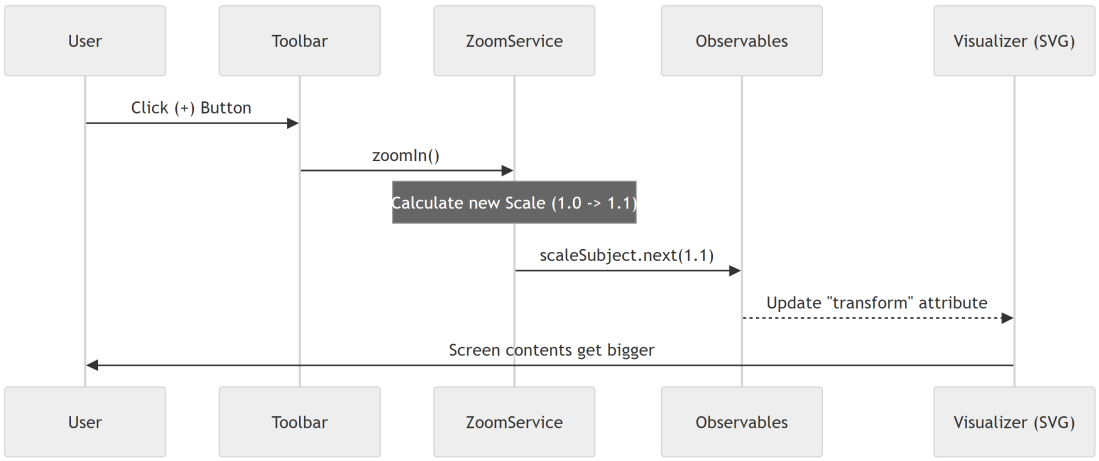

Figure 8.2: Sequence Diagram: Zoom In via Toolbar Button

8.4 Deep Dive: The Code

Let's explore `src/app/tr-services/zoom.service.ts` to see how we build this camera.

8.4.1 Storage (The State)

We use `BehaviorSubject` (special Observables) to hold the current camera position. This allows other parts of the app to always know where the camera is.

```
export class ZoomService {
    // Current Zoom level (starts at 1.0)
    private scaleSubject = new BehaviorSubject<number>(1);

    // Current Pan position (starts at 0,0)
    private offsetSubject = new BehaviorSubject<Point>({ x: 0, y: 0 });

    // Public streams that components can listen to
    public scale$ = this.scaleSubject.asObservable();
    public offset$ = this.offsetSubject.asObservable();
}
```

8.4.2 The Combined Output (`transform$`)

The visualizer doesn't want to do math. It just wants a string to put into the HTML. We check both `scale` and `offset` and combine them into one string automatically.

```
// Combines the two streams into one CSS string
public transform$: Observable<string> = combineLatest([
    this.scale$,
    this.offset$,
]).pipe(
    // Result: "translate(50 100) scale(1.5)"
    map(([scale, offset]) =>
        `translate(${offset.x} ${offset.y}) scale(${scale})`
    )
);
```

8.4.3 Smart Zooming (`zoomToPoint`)

When zooming, we don't just change the scale. We must also change the offset to keep the image stable. Imagine zooming towards a tree in a photo. The tree needs to stay in the same spot on your screen while the background expands.

```
private zoomToPoint(screenPoint: Point, targetScale: number): void {
    const currentOffset = this.currentOffset;
    const currentScale = this.currentScale;

    // Math formula: Keep the 'screenPoint' stationary
    const newOffsetX = screenPoint.x -
        (screenPoint.x - currentOffset.x) * (targetScale / currentScale);
    const newOffsetY = screenPoint.y -
        (screenPoint.y - currentOffset.y) * (targetScale / currentScale);

    // Update the state
    this.offsetSubject.next({ x: newOffsetX, y: newOffsetY });
    this.scaleSubject.next(targetScale);
}
```

8.4.4 Implementing "Fit Content"

This is the most complex function, but logically it is simple steps.

1. Get all Place/Transition positions.
2. Find the edges (Bounding Box).
3. Calculate how small the scale needs to be to fit that box in our window.

```
fitContent(viewportWidth: number, viewportHeight: number): void {
    // Step 1: Find the edges of the drawing
    const bbox = this.calculateContentBoundingBox();
    if (!bbox) return; // Empty graph

    const contentWidth = bbox.maxX - bbox.minX;

    // Step 2: Calculate scale ratio
    // Example: If content is 1000px and screen is 500px, scale is 0.5
    const scaleX = viewportWidth / contentWidth;
    let newScale = Math.min(scaleX, scaleY); // Fit both height and width

    // Step 3: Apply the new zoom
    this.scaleSubject.next(newScale);
    // (Offset calculation omitted for brevity)
}
```

8.5 Solving the Mouse Coordinate Problem

There is a catch!

If scale is `0.5` (zoomed out), and you move your mouse `100 pixels` on the screen, logically you have moved `200 pixels` in the zoomed-out world.

If we don't account for this, placing a new node would be inaccurate. The node would appear in the wrong spot.

We solve this with the **SvgCoordinatesService** (`src/app/tr-services/svg-coordinates-service.ts`). It injects the `ZoomService` to do the reverse math.

```
// Inside svg-coordinates-service.ts
getRelativeEventCoords(event: MouseEvent, drawingArea: HTMLElement): Point {
    // 1. Get raw pixel position from browser
    let rawX = event.clientX - svgRect.left;

    // 2. Adjust for the current Pan (Offset)
    // 3. Divide by Scale (Zoom)
    let scaledX = (rawX - this.zoomService.currentOffset.x)
                  / this.zoomService.currentScale;

    // Result: The logic coordinate exactly under the mouse
    return new Point(scaledX, scaledY);
}
```

8.6 Conclusion

The **ZoomService** acts as the lens of a camera.

- It tracks **Scale** and **Offset**.
- It generates the **CSS Transform** string for the visualizer.
- It ensures mouse clicks map to the correct logical coordinates, no matter how zoomed in or out you are.
- It powers the magical "Fit Content" button that saves users from getting lost in infinite space.

With this in place, our application is fully navigable. We can load data, see it, organize it, and move around it.

However, currently, we are checking "Which button is active?" (Place tool vs Move tool) using simple variables. As our application grows, we need a robust way to manage the state of the User Interface (Toolbars, Sidebars, Active Modes).

Next Chapter: UI State Manager (UiService)

Chapter 9

UI State Manager (UiService)

In the previous chapter, Camera Controller (ZoomService), we gave users the ability to pan and zoom around the canvas. We can now navigate the world we are building.

But a modern application is more than just a canvas. It has modes. You might be **Building** a net, **Simulating** a token game, or **Analyzing** the math behind it. You also have tools: a cursor might be a "Selector," a "Delete Tool," or a "Magic Wand."

How does the Toolbar talk to the Canvas? If you click "Delete" in the top menu, how does the canvas know that your next click should destroy an object?

We need a central "Remote Control." This is the **UiService**.

9.1 The Motivation: The "Remote Control"

Imagine you are watching TV. The TV screen (The Visualizer) displays the picture, but it doesn't decide *what* to show. You hold a remote control (The Toolbar) that tells the TV what to do.

In our application:

- **The Toolbar** sends signals (Channels/Volume).
- **The UiService** is the infrared beam carrying the signal.
- **The Visualizer** receives the signal and changes behavior.

9.1.1 The Use Case: Switching Modes

Goal: The user moves from the **"Build"** tab (where they draw circles) to the **"Simulation"** tab (where they watch tokens move).

Action: The drawing tools must disappear, and the "Play/Pause" buttons must appear.

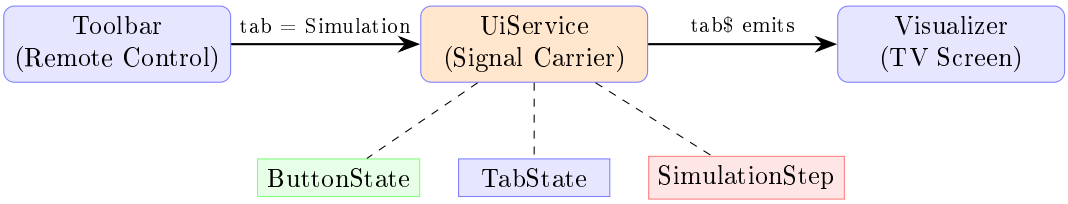

Figure 9.1: UiService as Central Remote Control between Toolbar and Visualizer

9.2 Key Concepts

9.2.1 Tab State (The TV Channel)

The application has major modes, which we call **Tabs**.

- **Build:** The edit mode.
- **Simulation:** The playback mode.
- **Analyze/Code:** Advanced modes.

Only one Tab can be active at a time. When the Tab changes, the entire interface rearranges itself.

9.2.2 Button State (The Active Tool)

Inside a specific channel, you have specific buttons.

- **Move:** Standard cursor.
- **Place:** Click to add a circle.
- **Delete:** Click to remove an item.

9.2.3 BehaviorSubjects (The Status LED)

Just like in previous chapters, we use **Observables**. Think of this as a status light on your dashboard.

- If the `tab$` light turns green, everyone knows we are in Simulation mode.
- If the `button$` light turns red, everyone knows the Delete tool is active.

9.3 Using the UiService

Let's see how components use this remote control to solve our "Switching Modes" use case.

9.3.1 Step 1: Changing the Channel (Toolbar)

When the user clicks a tab in the top menu (`ButtonBarComponent`), we update the service.

```
// Inside button-bar.component.ts
tabClicked(tabName: string) {
    if (tabName === 'simulation') {
        // Switch the channel to Simulation
        this.uiService.tab = TabState.Simulation;

        // Clear any specific tool being held
        this.uiService.button = null;
    }
}
```

9.3.2 Step 2: Listening for Signals (Visualizer)

The visualizer needs to know what mode we are in so it can hide or show helper lines.

```
// Inside a component constructor
constructor(public uiService: UiService) {}

// Inside the HTML template
<div *ngIf="uiService.tab === TabState.Build">
   <!-- Only show these tools if we are Building -->
   <button (click)="selectPlaceTool()">Add Place</button>
</div>
```

9.3.3 Step 3: Checking the Active Tool

When you click on the canvas, the visualizer asks the `UiService`: "What tool is the user holding?"

```
// Inside the visualizer, handling a mouse click
onCanvasClick() {
    // Check the 'button' property on the service
    if (this.uiService.button === ButtonState.Delete) {
        this.deleteObjectUnderMouse();
    } else {
        this.selectObjectUnderMouse();
    }
}
```

9.4 Under the Hood: The Logic

The `UiService` is a purely logical state container. It does not manipulate the DOM or draw SVG. It simply holds variables and shouts when they change.

9.4.1 Sequence Diagram: Creating a Place

Here is the flow of information when a user selects the "Add Place" tool and clicks the canvas.

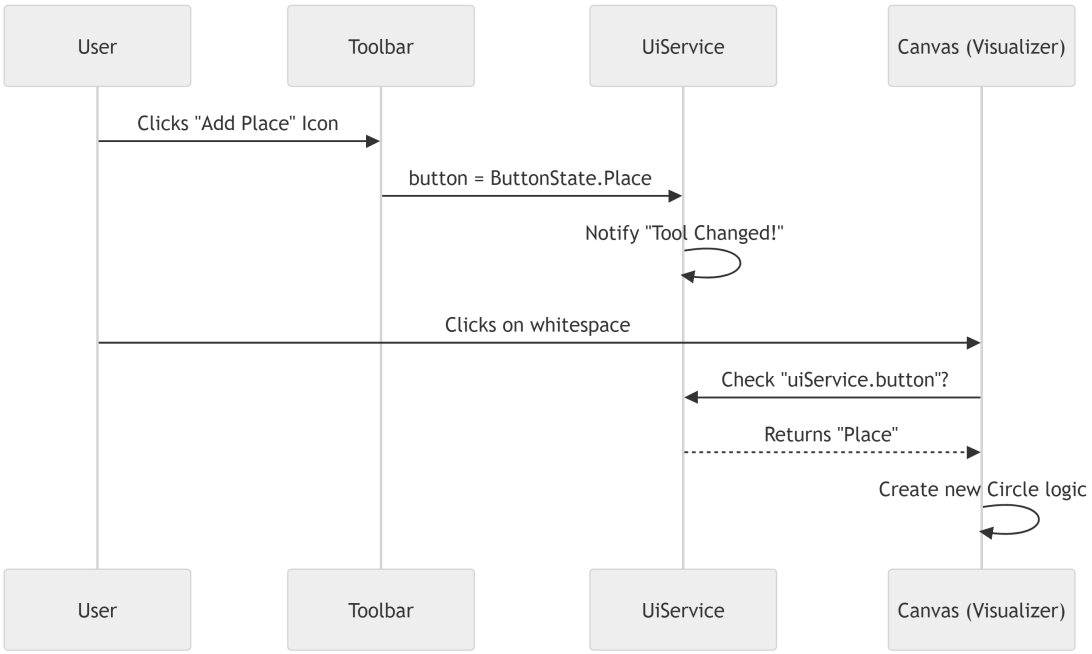

Figure 9.2: Sequence Diagram: Tool Selection and Place Creation

9.5 Deep Dive: The Code

Let's look at `src/app/tr-services/ui.service.ts` to see how this simple state machine is built.

9.5.1 Managing Tabs (The Channels)

We use a private variable `_tab` to store the value, and a `BehaviorSubject` to broadcast changes.

```
export class UiService {
    // 1. Default start state is 'Build'
    private _tab: TabState = TabState.Build;

    // 2. The Broadcast system
    private _tab$ = new BehaviorSubject<TabState>(this._tab);
    public tab$ = this._tab$.asObservable();

    // 3. Setter: When you change the variable, we notify subscribers
    public set tab(value: TabState) {
        this._tab = value;
        this._tab$.next(value); // Broadcast!
    }
}
```

9.5.2 Managing Simulation Steps (The Timeline)

The `UiService` also acts as the scoreboard for the Simulation. It tracks which "Step" (or frame) of the movie we are currently watching.

```
// Tracks current progress (0 = Start, 10 = Step 10)
private _currentSimulationStep$ = new BehaviorSubject<number>(0);

// Public method to move the timeline
setCurrentSimulationStep(step: number): void {
    this._currentSimulationStep$.next(step);
}

// Public getter
getCurrentSimulationStep(): number {
    return this._currentSimulationStep$.getValue();
}
```

9.5.3 Simulation Modes (Auto vs Manual)

This service also tracks if the simulation is playing automatically (like a video) or if the user is manually clicking tokens (like a game).

```
// 'automatic' or 'manual'
private _simulationMode$ = new BehaviorSubject<SimulationMode>('automatic');

setSimulationMode(mode: SimulationMode): void {
    // If switching TO manual, remember where we paused
    if (mode === 'manual') {
        this.lastAutomaticStep = this.getCurrentSimulationStep();
    }
    this._simulationMode$.next(mode);
}
```

Why save the step? If you are watching a simulation and pause it to click around manually, you want the "Resume" button to pick up exactly where you left off, not restart from the beginning.

9.6 Conclusion

The **UiService** connects the disparate parts of our application.

- It allows the **Toolbar** to influence the **Canvas**.

- It allows different components to stay in sync (e.g., if you switch tabs, all tools reset).
- It acts as the single source of truth for "What is the user doing right now?"

Now that we have a fully controllable interface—we can load files, zoom, move nodes, and switch modes—it is time to implement the most complex feature of a Petri net application: **The Logic.**

We need a service that understands the rules of the Token Game. It needs to calculate which transitions can fire and move the tokens from Place A to Place B.

Next Chapter: Simulation Engine (TokenGameService)

Chapter 10

Simulation Engine (TokenGameService)

In the previous chapter, UI State Manager (UiService), we built the "Remote Control" for our application. We can switch between "Build Mode" and "Simulation Mode."

However, switching to Simulation Mode currently just changes the toolbar buttons. If you click on a Transition, nothing happens. The tokens sit still. The application has no idea how to actually play the game.

We need a **Rulebook**. We need a service that enforces the mathematical logic of Petri nets. This is the **TokenGameService**.

10.1 The Motivation: The Rulebook

Imagine you are playing a board game like Chess or Monopoly.

- **The Board:** Petri Net Primitives (Places/Transitions).
- **The Pieces:** Tokens.
- **The Player:** You.

But who tells you if a move is legal? Who remembers where the pieces were before you knocked the board over?

The **TokenGameService** has two jobs:

1. **The Referee:** It decides if a Transition is allowed to "Fire" (execute).
2. **The Historian:** It records every move so you can "Undo" if you make a mistake.

10.1.1 The Use Case: "Firing" a Transition

- **Scenario:** We have a Place (Water) with 1 token connected to a Transition (Brew).
- **Action:** The user clicks the "Brew" transition.
- **Logic:** The service checks: "Do we have water?" (Yes). It subtracts 1 token from Water and adds 1 token to Coffee.
- **Undo:** The user clicks "Step Back." The token returns to the Water tank.

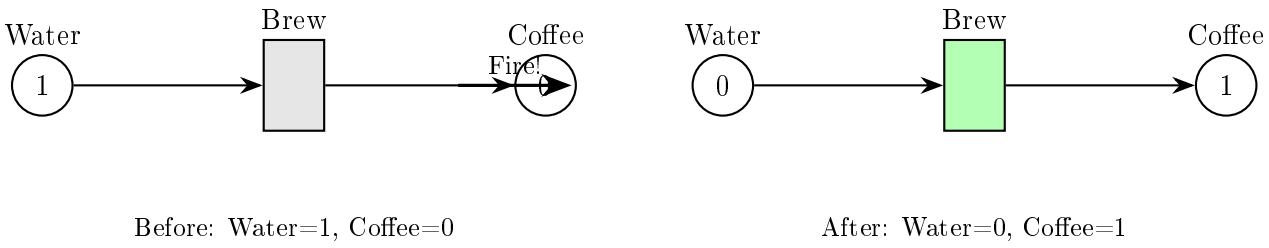

Figure 10.1: Firing a Transition: Tokens move from input to output places

10.2 Key Concepts

10.2.1 Firing

In Petri net terms, **Firing** means a transition executes.

1. **Consume:** It removes tokens from input places.
2. **Produce:** It adds tokens to output places.

10.2.2 The "Game State"

The state of a Petri net is simply a snapshot of **where the tokens are right now**. It is a list:

- Place A: 5 tokens
- Place B: 0 tokens
- Place C: 1 token

10.2.3 History (The Stack)

To enable "Undo," we use a **Stack** (Last-In, First-Out).

- Every time you make a move, we take a photo of the board (Current State) and put it on top of a pile.
- When you click **Undo**, we take the top photo and arrange the board to match it.

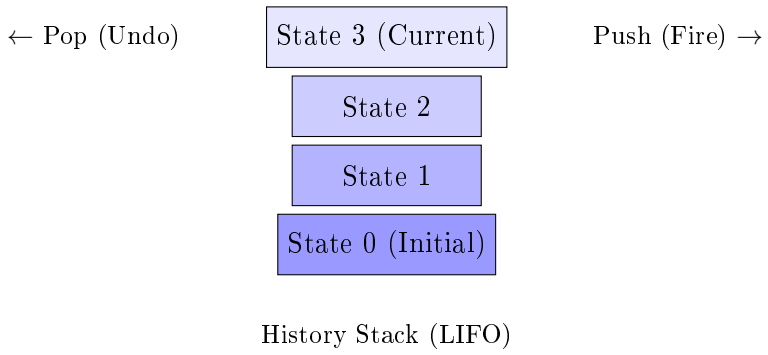

Figure 10.2: History Stack enables Undo functionality

10.3 Using the Service

Let's see how we use this engine to move tokens.

10.3.1 Step 1: Checking Rules

Before firing, we usually want to know if it's even possible.

```
// Inside a component
checkIfCanFire(transition: Transition) {
    // The Transition class itself knows if it has enough tokens
    if (transition.isActive) {
        console.log("Ready to brew!");
        // We can safely call the service
        this.tokenGameService.fire(transition);
    }
}
```

10.3.2 Step 2: Firing (The Move)

When the user clicks, we tell the engine to execute the math.

```
// Perform the move
this.tokenGameService.fire(myTransition);

// Note: This modifies the 'token' counts inside the DataService directly.
// The Visualizer will automatically update because it watches the DataService.
```

10.3.3 Step 3: Time Travel (Undo)

If the user wants to go back.

```
// Go back one step
this.tokenGameService.revertToPreviousState();

// Reset to the very beginning (restart game)
this.tokenGameService.resetGame();
```

10.4 Under the Hood: The Flow

What happens when you click a transition? The service performs a "Save, then Update" operation.

10.4.1 Sequence Diagram: Firing Logic

Here is the lifecycle of a single move.

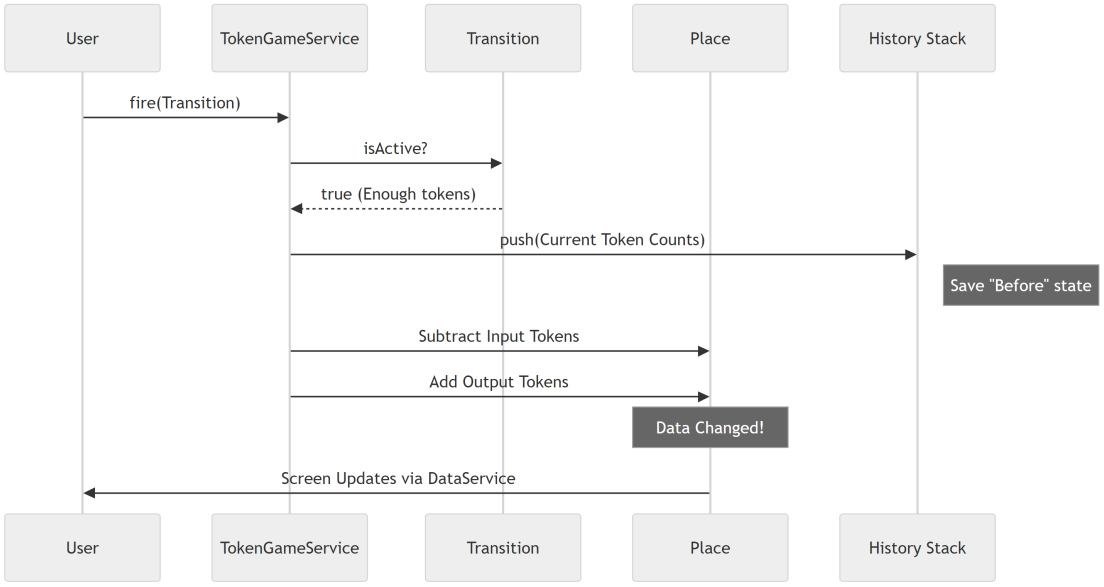

Figure 10.3: Sequence Diagram: Token Game Firing Logic

10.5 Deep Dive: The Code

Let's look into `src/app/tr-services/token-game.service.ts`.

10.5.1 The History Storage

We need to remember the token count for every single place. We use a `Map` where the key is the `Place` object, and the value is the `number` of tokens.

```
export class TokenGameService {
    // A stack (array) of states.
    // Each state is a Map of Place -> TokenCount
    private _tokenHistory: Map<Place, number>[] = [];

    constructor(protected dataService: DataService) {}
    // ...
}
```

10.5.2 Capturing a Snapshot

Before we change anything, we walk through every place in the Central Data Store (DataService) and write down how many tokens it has.

```
private getGameState(): Map<Place, number> {
    const tokenMapping = new Map<Place, number>();

    // Loop through all existing places
    for (let place of this.dataService.getPlaces()) {
        // Save the pair: [Place Object, 3 tokens]
        tokenMapping.set(place, place.token);
    }
    return tokenMapping;
}
```

10.5.3 The Fire Method (The Core Logic)

This is the heart of the engine. Notice how it saves the state *before* doing the math.

```
fire(transition: Transition) {
    // 1. Check the rule
    if (transition.isActive) {

        // 2. Save history for Undo
        this.saveCurrentGameState();

        // 3. Subtract tokens from inputs (Pre-places)
        for (let arc of transition.preArcs) {
            // Note: arc.weight is usually stored as negative for inputs
            (arc.from as Place).token += arc.weight;
        }

        // 4. Add tokens to outputs (Post-places)
        for (let arc of transition.postArcs) {
            (arc.to as Place).token += arc.weight;
        }
    }
}
```

Why `+= arc.weight` for subtraction? In Chapter 1, we learned that input arcs store their weight as a negative number (e.g., -1). So $5 + (-1) = 4$. This makes the math elegantly simple.

10.5.4 Implementing Undo (`revertToPreviousState`)

To undo, we pop the last snapshot off the stack and force the current places to match those numbers.

```
revertToPreviousState() {
    // 1. Get the last saved photo
    const state = this._tokenHistory.pop();

    // 2. If history is empty, do nothing
    if (!state) return;

    // 3. Overwrite current reality with the saved photo
    this.setGameState(state);
}
```

10.5.5 Applying the State

This helper function takes a snapshot and applies it to the living objects.

```
private setGameState(state: Map<Place, number>) {
    for (let place of this.dataService.getPlaces()) {
        const tokens = state.get(place);

        // Safety check: ensure the place still exists
        if (tokens !== undefined) {
            place.token = tokens;
        }
    }
}
```

10.6 Conclusion

The **TokenGameService** brings our static diagrams to life.

- It acts as the **Referee**, ensuring only valid moves occur.
- It acts as the **Time Machine**, saving snapshots of the world before every move so we can undo them.
- It directly manipulates the tokens in the `DataService`, which causes the `PetriNetComponent` to redraw the screen instantly.

We now have a fully functional application! We can draw, edit, zoom, save, and manually play the token game.

But... manual playing is slow. What if we want to ask a computer: "Is there a path from State A to State B?" or "Find the shortest route to finish the process"? For this, we need to connect our frontend to a powerful backend algorithm.

Next Chapter: Backend Integrator (PlanningService)

Chapter 11

Backend Integrator (PlanningService)

In the previous chapter, Simulation Engine (TokenGameService), we built a logic engine that runs directly inside your browser. It allows you to manually click transitions and play the "Token Game."

But there is a limit. Your browser is great for drawing and simple logic, but it isn't a supercomputer.

What if you need to simulate a workflow 10,000 times to maximize efficiency? What if you need to calculate complex mathematical reachability graphs? If we tried to do this in the visualization loop, your browser would freeze and crash.

We need to send this heavy work to a dedicated server. We need the **PlanningService**.

11.1 The Motivation: The "Phone Line"

Think of your application like a **Restaurant**.

- **The Frontend (Your App):** This is the Dining Area. It looks nice, handles menus (UI), and seats guests (Layout).
- **The Backend (Server):** This is the Kitchen. It is noisy, hot, and full of heavy machinery to cook the food.

The Waiter (Frontend) doesn't cook the steak at the table. The Waiter takes the order, sends it to the kitchen, and waits for the finished plate to come back.

The **PlanningService** is that connection between the Dining Area and the Kitchen.

11.1.1 The Use Case: "Heavy Simulation"

Goal: The user has drawn a complex Offshore Wind Farm logistics model. They want to know: "If I run this for 1 year, how much money do I lose?"

1. **Action:** User clicks "Run Simulation."
2. **Process:** The app bundles the Petri net into a file, uploads it to the backend server, and waits.
3. **Result:** The server returns a JSON report with raw data, which the frontend displays.

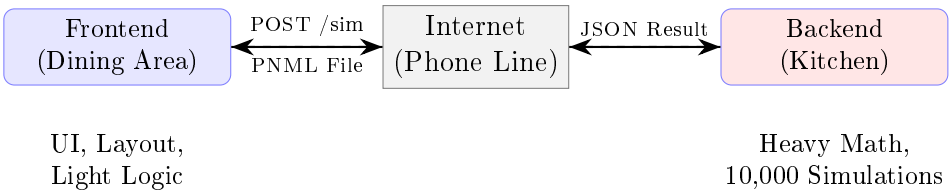

Figure 11.1: PlanningService: The Phone Line between Frontend and Backend

11.2 Key Concepts

11.2.1 HTTP Requests (GET and POST)

This is how the web talks.

- **GET:** "Hey Server, give me the default settings." (Like asking for a menu).
- **POST:** "Hey Server, here is a file. Process it and tell me the answer." (Like submitting an order).

11.2.2 Asynchronous (The Waiting Game)

When you order food, you don't freeze like a statue until it arrives. You keep chatting. Similarly, our requests are **Asynchronous**. We send the request, let the user keep moving the mouse, and when the server replies 2 seconds later, we handle the result.

11.2.3 FormData (The Envelope)

To send a file (like our `.pnml` model) across the internet, we can't just send text. We wrap it in a special digital envelope called `FormData`. This allows us to attach files just like an email attachment.

11.3 Using the Integrator

Let's look at how we solve the "Heavy Simulation" use case.

11.3.1 Step 1: Preparing the Data

First, we need the text of our Petri net. We use the logic from PNML/XML Translator (PnmlService) to generate the string.

```
// Inside a component (e.g., ParameterInputComponent)
const pnmlString = this.pnmlService.writePNMLToString();

// Define how many times to run the sim
const runs = 100;
```

11.3.2 Step 2: Calling the Service

We inject the `PlanningService` and ask it to send the data. Notice we are sending a **String**, and the service converts it to a file automatically.

```
// Call the method
this.planningService.runSimpleSimulationFromString(pnmlString, runs)
    .subscribe({
        next: (result) => {
```

```
            console.log("Success! The server replied:", result);
            this.showResults(result);
        },
        error: (err) => {
            console.error("The kitchen is on fire!", err);
        }
    });
```

What happens? The `.subscribe()` part is crucial. It tells the app: "Send the message now, and wake me up when the answer arrives."

11.4 Under the Hood: The Communication Flow

What happens in those few seconds between clicking "Run" and seeing the result?

1. **Frontend:** Wraps the XML string into a virtual file.
2. **Network:** Travels across the internet to the URL (e.g., `api/simulation`).
3. **Backend:** The Java/Python server parses the file, runs the heavy math, and generates a JSON response.
4. **Frontend:** Receives the JSON and updates the UI status message.

11.4.1 Sequence Diagram: Running a Simulation

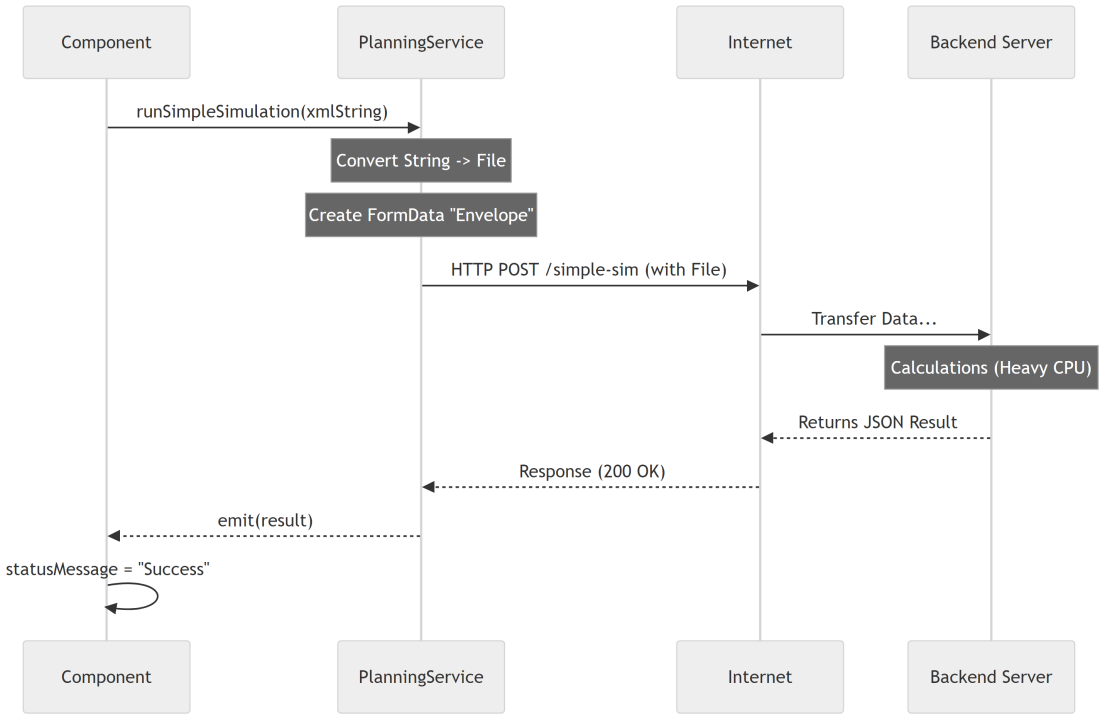

Figure 11.2: Sequence Diagram: Running a Backend Simulation

11.5 Deep Dive: The Code

Let's look at `src/app/tr-services/planning.service.ts` to see how the "Phone Line" is constructed.

11.5.1 Setup and Environment

We don't hardcode the server address (like `localhost`). We use an environment variable so the app works in development and production.

```
@Injectable({ providedIn: 'root' })
export class PlanningService {
    // Construct the phone number from configuration
    private simpleSimApiUrl =
        `${environment.backendApiUrl}/simulation-petri-nets/simple-sim`;

    constructor(private http: HttpClient) { }
}
```

11.5.2 The Simulation Call (`runSimpleSimulation`)

This is the most important method. It prepares the "Envelope" (`FormData`). It accepts a real JavaScript `File` object.

```
runSimpleSimulation(pnmlFile: File, runs: number = 1): Observable<any> {
    // 1. Create the envelope
    const formData = new FormData();

    // 2. Stuff the file and parameters inside
    formData.append('pnml_model', pnmlFile, pnmlFile.name);
    formData.append('num_runs', runs.toString());

    // 3. Send via HTTP POST
    return this.http.post<any>(this.simpleSimApiUrl, formData)
        .pipe(
            // 4. Attach a safety net for errors
            catchError(this.handleError)
        );
}
```

11.5.3 The Helper (`runSimpleSimulationFromString`)

Often, the file doesn't exist on the user's hard drive; it exists in the browser's memory (because the user just drew it). This helper converts the string to a file on the fly.

```
runSimpleSimulationFromString(content: string, runs: number = 1): Observable<any
    > {
    // Create a virtual file from the text string
    const blob = new Blob([content], { type: 'application/xml' });
    const file = new File([blob], 'current_model.pnml');

    // Reuse the main method
    return this.runSimpleSimulation(file, runs);
}
```

11.5.4 Fetching Defaults (`getDefaults`)

Sometimes we just need to read data (GET), like asking the server "What are valid parameters?".

```
getDefaults(): Observable<any> {
    // Simple GET request
    return this.http.get<any>(`${this.apiUrl}/defaults`)
        .pipe(catchError(this.handleError));
}
```

11.5.5 Error Handling (handleError)

If the server crashes or the internet disconnects, we don't want the user to see a blank screen. We catch the error and format a nice message.

```
private handleError(error: HttpErrorResponse) {
    let msg = 'Unknown Error';

    if (error.status === 0) {
        msg = 'Network Error: Is the server running?';
    } else {
        msg = `Server Error (Code ${error.status}): ${error.message}`;
    }

    console.error(msg);
    return throwError(() => new Error(msg));
}
```

11.6 Integration: The "Parameter Input" Form

Included in the project is `parameter-input.component.ts`. This is the UI form where the user types "Runs: 100".

It connects to the `PlanningService` in its `runSimulation` method.

```
// Inside parameter-input.component.ts

runSimulation(): void {
    this.isLoadingSimulation = true;

    // 1. Call the service
    this.planningService.runPlanning(planningData)
        .pipe(finalize(() => {
            // This runs whether success or fail (turn off spinner)
            this.isLoadingSimulation = false;
        }))
        .subscribe({
            next: (response) => {
                this.statusMessage = "Simulation Complete!";
            },
            error: (err) => {
                this.statusMessage = "Error: " + err.message;
            }
        });
}
```

What is `finalize`? It's a handy operator that ensures your "Loading..." spinner stops spinning, even if the request executes successfully or fails miserably.

11.7 Conclusion

The **PlanningService** is the gateway to power.

- It handles **HTTP Communication** with the backend.
- It manages **File Uploads** via `FormData`.
- It provides **Observables** so the UI can stay responsive while waiting.
- It enables **Isomorphic Simulation**: We can run simple tests in the browser (TokenGameService) or massive simulations on the server using the exact same logical model.

Congratulations! You have completed the **PNML_Frontend** manual series.
You now understand the full stack of the application:

1. **Primitives:** The atoms (Place, Transition, Arc).
2. **DataService:** The central brain.
3. **Visualizer:** The artist.
4. **PnmlService:** The translator.
5. **Graph Layout:** The architect.
6. **Interaction/Zoom:** The controls.
7. **Ui/TokenGame:** The logic.
8. **PlanningService:** The bridge to the backend.

You are now ready to extend the application, add new simulation types, or design your own custom Petri net tools!